

CareerConnect AI Usage Log

Sprint 1
Nazila Hoche Abdi

Task 1.SCRUM-17: Implement Database (Connection & Initial Models)

Purpose of AI Use	Coding: implementing the MongoDB connection and initial Mongoose data models.
--------------------------	---

Prompt:

Asked the AI for a full beginner walkthrough to implement SCRUM-17: setting up a MongoDB Atlas database, connecting it to a Node.js project using Mongoose, creating data models for the app's core entities (User, Resume, JobPosting, Application), and testing basic CRUD operations.

Response:

The AI provided a connectDB() function using Mongoose and an environment variable for the connection string, four Mongoose schemas, and a standalone test script to verify a create-and-read cycle. Example of the connection code provided:

```
const mongoose = require("mongoose");

const connectDB = async () => {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log("MongoDB connected successfully");
  } catch (error) {
    console.error("MongoDB connection failed:", error.message);
    process.exit(1);
  }
};

module.exports = connectDB;
```

AI-Suggested Content	A MongoDB connection module (config/db.js), four Mongoose models (User, Resume, JobPosting, Application), and a testDb.js script to verify the connection independently of the server.
Validation	Ran node testDb.js repeatedly after each change; confirmed a successful connection, a created user document with correct field values, and a matching document retrieved by ID.
Decision	Modified before use, the User model was adjusted so only email and password are required, matching the actual scope of the team's Sprint 1 /api/register mock endpoint.
Reflection	AI guidance was effective for scaffolding an unfamiliar technology (MongoDB/Mongoose) from zero prior experience. It was also useful for debugging: when a model failed with "X is not a function," the AI suggested reading the raw file contents directly from disk, which revealed the file had not actually saved despite appearing correct in the editor.
Responsible Person	Nazila Hoche Abdi

Task 1.SCRUM-25: DEV - Setup Database (Queries Folder and Initial Queries)

Purpose of AI Use	Requirements clarification and coding and understanding what a 'database query' function is, then implementing a reusable queries layer.
--------------------------	--

Prompt:

Explained that the assigned task was 'Set up database folder structure and initial queries' but that the term 'queries' was unclear; asked the AI to explain the concept in simple terms and then implement it for the User, JobPosting, and Application models.

Response:

The AI explained that a query is a reusable function that performs one specific database operation (e.g., 'find a user by email'), so that other team members can call a simple function instead of writing raw Mongoose code themselves. It then provided a queries/ folder with one file per model. Example (queries/userQueries.js):

```
const User = require("../models/User");

async function createUser({ email, password, name, role }) {
  return await User.create({ email, password, name, role });
}

async function findUserByEmail(email) {
  return await User.findOne({ email });
}

module.exports = { createUser, findUserByEmail, /* ...other functions */ };
```

AI-Suggested Content	Three query files (userQueries.js, jobPostingQueries.js, applicationQueries.js), each exporting reusable create/find/update/delete functions, plus an updated testDb.js that tests through the new functions instead of calling the model directly.
Validation	Ran node testDb.js after implementing the query functions; confirmed a user was created and found successfully through createUser() and findUserByEmail() rather than direct model calls.
Decision	Accepted.
Reflection	This session repeated the same 'file appears correct in the editor but is empty on disk' bug from Task 1.SCRUM-17. The AI's diagnostic approach (reading the file directly via Node's fs module, bypassing the editor) again correctly identified the cause. This reinforced the importance of verifying a save independently rather than trusting the editor's visual state.
Responsible Person	Nazila Hoche Abdi

Task 1.SCRUM-103: Extract Resume Database Queries

Purpose of AI Use	Coding/refactoring: bringing the resume feature in line with the team's existing query-layer pattern.
--------------------------	---

Prompt:

Asked the AI to extract the Resume.findOne/create/findOneAndUpdate/findOneAndDelete calls, currently written inline in a teammate's resumeRoutes.js, into a separate queries/resumeQueries.js file, matching the pattern already used for User, JobPosting, and Application.

Response:

The AI provided a new resumeQueries.js file and the corresponding updated resumeRoutes.js with the same behavior, just calling the new functions instead of the model directly.

```
const Resume = require("../models/Resume");

async function findResumeByUser(userId) {
  return await Resume.findOne({ user: userId });
}

async function replaceResume(userId, data) {
  return await Resume.findOneAndUpdate(
    { user: userId },
    { user: userId, ...data },
    { new: true, upsert: true, runValidators: true }
  );
}

module.exports = { findResumeByUser, replaceResume, /* ...other functions */ };
```

AI-Suggested Content	queries/resumeQueries.js with five functions (findResumeByUser, findResumeMetaByUser, createResume, replaceResume, deleteResumeByUser), and an updated resumeRoutes.js using them.
Validation	Restarted the local server and re-ran the full curl CRUD cycle (GET, PUT, DELETE, POST) with a fresh JWT; confirmed identical behavior and response data to before the refactor.
Decision	Accepted.
Reflection	Since this PR was built on top of a teammate's still-unmerged branch, the AI also correctly flagged the need to target the pull request's base branch at the teammate's branch rather than main, avoiding a confusing diff and a premature merge conflict.
Responsible Person	Nazila Hoche Abdi

Task 1.SCRUM-87: Test Database (Verify Database Functionality)

Purpose of AI Use	Testing, verifying that new Sprint 1 functionality (resume storage and retrieval) persists correctly in MongoDB, per the parent 'Test Implementation' story.
--------------------------	--

Prompt:

Asked the AI how to confirm the resume feature's data was actually present in MongoDB (not just returned successfully by the API), and separately how to test that a stored resume file could be downloaded back out as a valid, working PDF.

Response:

The AI guided checking the correct database/collection in the Atlas UI (identifying that data had landed in a default 'test' database rather than a named one, due to the connection string), and provided a curl command using the -o flag to save the /file route's response directly to disk for inspection.

```
curl.exe http://localhost:3000/api/resumes/file \
-H "Authorization: Bearer <token>" \
-o downloaded-resume.pdf
```

AI-Suggested Content	Steps to locate the correct database in Atlas, a method to list collections and document counts directly from Node as a sanity check, and the curl-based file download test above.
Validation	The downloaded file matched the original upload's exact byte size (102,151 bytes) and opened successfully as a valid, readable PDF, confirming the binary data survived the full store-and-retrieve cycle intact.
Decision	Accepted.
Reflection	This test surfaced a genuine infrastructure issue: a MongoDB Atlas SSL/connection error caused by the local IP address falling off the cluster's network access whitelist after a network change, and separately, an expired JWT test token being reused by mistake. Both were real, legitimate bugs to catch during end-of-sprint testing rather than assuming a single earlier successful test meant everything still worked.
Responsible Person	Nazila Hoche Abdi

Task 1.SCRUM-23: Review and Test Resume Upload and Management (Teammate PR)

Purpose of AI Use	Code review and testing/debugging: reviewing a teammate's completed resume upload feature and validating it against the real database.
--------------------------	--

Prompt:

Shared a teammate's PR (Resume.js model, resumeRoutes.js, and the relevant server.js changes) and asked the AI to review the code and guide a full local test of the upload/view/replace/delete functionality against the live MongoDB Atlas database.

Response:

The AI reviewed the model design (binary file storage in MongoDB vs. an earlier fileUrl-based design), flagged a legitimate security note already raised by a Copilot review comment (MIME type spoofing), and confirmed the server integration reused the existing config/db.js with no conflicts. It then guided testing the feature end-to-end using curl, including generating a manual JWT token to satisfy authentication middleware that had no login endpoint built yet.

AI-Suggested Content	A full review of the model/routes/server changes, and a step-by-step local testing procedure: starting the server, generating a test JWT via jsonwebtoken.sign(), and exercising GET, POST, PUT, and DELETE routes with curl, then verifying results directly in MongoDB Atlas.
Validation	All four routes were tested against the real database and returned correct results (successful upload with metadata, duplicate-upload 409 error, successful replace, and successful delete with 204). Data was also visually confirmed in the MongoDB Atlas collection viewer.
Decision	Accepted (PR approved after testing).
Reflection	This was the most extensive debugging session of the sprint. The AI helped methodically resolve a series of real, unrelated issues: Node.js/npm not being installed, a malformed .env file (missing the MONGO_URI= key), Windows PowerShell's curl alias not supporting -X, a OneDrive file sync issue blocking file reads, and eventually a MongoDB Atlas IP whitelist/TLS connection failure caused by a network change. Each issue was diagnosed by checking one specific piece of evidence at a time rather than guessing, which was a useful debugging habit to observe and repeat independently going forward.
Responsible Person	Nazila Hoche Abdi

Note: This log documents AI-assisted activity for database development and testing during Sprint 1.
Team-generated ideas and modifications are recorded separately per assignment instructions.